

KAN-33

Block "" - Mostrar rating promedio (Backend)

- Implementar lógica backend para calcular y retornar el promedio de calificaciones de cada mentor.
- Permitir mostrar métricas de reputación y valoraciones acumuladas.

- - - - -

El objetivo principal de este bloque consiste en implementar la lógica de cálculo dinámico para el promedio de calificaciones de los mentores en el backend, permitiendo exponer métricas de reputación precisas dentro del ecosistema de perfiles. El sistema procesa las valoraciones asociadas a cada mentor mediante funciones de agregación optimizadas, cumpliendo con las siguientes reglas de negocio:

- **Cálculo Agregado y Eficiente:** Se prioriza el procesamiento a nivel de base de datos utilizando funciones de agregación SQL, evitando la sobrecarga innecesaria de memoria en el servidor PHP al no cargar todas las valoraciones individuales.
- **Precisión y Formato:** La métrica resultante se obtiene y expone con la precisión necesaria para ser consumida directamente por la interfaz, integrándose de forma nativa en la respuesta JSON de los perfiles.
- **Integridad de Datos:** La implementación asegura que los perfiles sin valoraciones previas se mantengan visibles en el listado, gestionando correctamente la ausencia de métricas (retornando un valor nulo o cero).
-

El desarrollo de este requerimiento se concentró en los siguientes componentes:

- **app/Models/Perfil.php (Relación):** Se definió la relación `hasMany` hacia el modelo `Valoracion` utilizando la clave foránea `mentor_id`. Esta relación es el pilar que permite la navegabilidad y las consultas agregadas.
- **app/Http/Controllers/PerfilController.php (Lógica de Consulta):** Se integró el método `withAvg` en el endpoint `index`, realizando un `LEFT JOIN` optimizado que inyecta automáticamente el campo `valoraciones_avg_calificacion` en cada objeto de perfil. Además, se añadió este campo a los parámetros permitidos para ordenamiento dinámico, permitiendo al frontend filtrar por reputación.

Configuración y Seguridad: La implementación aprovecha la robustez de Eloquent ORM para delegar el cálculo al motor de base de datos, garantizando que la consulta permanezca eficiente incluso ante un crecimiento considerable del volumen de valoraciones.

La verificación del sistema se realizó mediante:

- **Validación de API (Postman):** Pruebas de consumo del endpoint `GET /api/perfiles` confirmando la correcta inyección del campo `valoraciones_avg_calificacion`.
- **Pruebas de Ordenamiento:** Verificación de la funcionalidad de ordenamiento dinámico mediante parámetros de query string, asegurando que la reputación influye correctamente en la presentación de los mentores.